



**ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ**
UNIVERSITY OF PATRAS

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

ΟΝΟΜΑΤΕΠΩΝΥΜΟ ΦΟΙΤΗΤΗ: ΣΑΛΙΑΡΗΣ ΓΕΩΡΓΙΟΣ

ΑΜ: 1092584

ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ: 2024-2025

ΜΑΘΗΜΑ: ΕΙΣΑΓΩΓΗ ΣΤΗ ΡΟΜΠΟΤΙΚΗ

**ΥΠΕΥΘΥΝΟΣ ΚΑΘΗΓΗΤΗΣ: ΚΩΝΣΤΑΝΤΙΝΟΣ
ΧΑΤΖΗΛΥΓΕΡΟΥΔΗΣ**

ΑΤΟΜΙΚΗ ΕΡΓΑΣΙΑ #4

A. Σκοπός / Περιγραφή της εργασίας

Η εν λόγω εργασία αν και συνδεδεμένη σε ένα βαθμό με τις προηγούμενες 3 εργασίες, αφορά ένα διαφορετικό (απλούστερο) σύστημα. Ειδικότερα, μας δίνεται η περιγραφή -μέσω ενός URDF αρχείου- ενός ρομποτικού βραχίονα ο οποίος αποτελείται από 4 *links*. Αφού αντλήσουμε τις απαραίτητες πληροφορίες από το αρχείο αυτό καλούμαστε να κάνουμε το forward dynamics simulator για τον συγκεκριμένο *manipulator*. Δηλαδή, χωρίς την επίδραση εξωτερικών δυνάμεων, θέλουμε να δούμε την κατάσταση του ρομποτικού χεριού. Εν συνεχεία, επιθυμώντας το τελικό στοιχείο δράσης να ακολουθήσει μια ημιτονοειδή τροχιά, σχεδιάζουμε έναν task-space controller, στόχος του οποίου είναι να καταστήσει την <<πραγματική>> τροχιά του *end-effector*, όσο το δυνατό πιο κοντά στη ζητούμενη.

B. Ζητούμενα της άσκησης

Ερώτημα 1 : Εξαγωγή πληροφοριών για την Κινηματική και τη Δυναμική του βραχίονα

Αρχικά να πούμε ότι κάθε *body/joint frame* καθορίζεται σε σχέση με το αντίστοιχο *parent frame*. Το CoM καθορίζεται σε σχέση με το *εκάστοτε link frame*. Επίσης, το RPY (*Roll-Pitch-Yaw*) δηλώνει αν έχουμε περιστροφή του *εκάστοτε frame* ως προς τους άξονες x,y,z αντιστοίχως του σχετικού *parent frame* (επομένως κάνουμε τις απαραίτητες προθέσεις προκειμένου να βρούμε το *εκάστοτε body frame* ως προς το *world frame*). Παρατηρούμε ότι σύμφωνα με το αρχείο δεν έχουμε καμία περιστροφή, επομένως στους σχετικούς πίνακες μετασχηματισμού το rotational part θα είναι ίδιο με τον 3x3 μοναδιαίο πίνακα σε όλες τις περιπτώσεις, γι' αυτό το λόγο επομένως δεν αναφέρεται παρακάτω.

Επομένως, για τα *body frames* όπως επίσης και για τα *CoM frames* για όλα τα *links* έχουμε τα εξής (παρατήρηση: θεωρούμε ότι το *world frame* είναι το *body frame* του *arm_link_0*, ειδάλλως προσθέτουμε +0.02725 σε κάθε z συντεταγμένη όλων των *frames* τα οποία έπονται

Πίνακας 1

Link name	Body frame position (relative to the world frame) [m]	CoM Frame Position [m]
arm_link_0	(0 0 0)-base frame	—
arm_link_1	(0 0 0.0545)	(0,0,0.116)
arm_link_2	(0 0 0.1775)	(0,0,0.336)
arm_link_3	(0 0 0.4945)	(0,0,0.5955)
arm_link_4	(0,0,0.6965)	(0,0,0.77675)
arm_link_5	(0,0,0.857)	—

Για τα *joint frames* καταλαβαίνουμε και βάσει της εικόνας του ρομποτικού βραχίονα που μας δίνεται αλλά και βάσει λογικής ότι το $joint_{frame_i}$ θα είναι το ίδιο με το $link_{frame_{i-1}}$. Άρα:

Πίνακας 2

Joint name	Joint frame position (relative to the world frame) in meters
arm_joint_1	(0,0,0.0545)
arm_joint_2	(0,0,0.1775)
arm_joint_3	(0,0,0.4945)
arm_joint_4	(0,0,0.6965)
arm_joint_5	(0,0,0.857)

Για τη δε μάζα κάθε σώματος θα έχουμε:

Πίνακας 3

Link	Mass [kg]
1	0.190421352
2	0.29302326
3	0.21931466
4	0.15813986
5	—

Και για το *rotational inertia matrix* για κάθε *body (link)*, ο οποίος καθορίζεται ως προς το *frame* του αντίστοιχου *link*:

Body 1

$$I_1 = \begin{bmatrix} i_{xx} & i_{xy} & i_{xz} \\ i_{yx} & i_{yy} & i_{yz} \\ i_{zx} & i_{zy} & i_{zz} \end{bmatrix}$$

$$= \begin{bmatrix} 0.000279744834534 & 0 & 0 \\ 0 & 0.000265717763008 & 0 \\ 0 & 0 & 6.53151584738e-05 \end{bmatrix} \quad (1)$$

Body 2

$$I_2 = \begin{bmatrix} 0.00251484771035 & 0 & 0 \\ 0 & 0.00248474836108 & 0 \\ 0 & 0 & 9.19936757328e-05 \end{bmatrix} \quad (2)$$

Body 3

$$I_2 = \begin{bmatrix} 0.000791433503053 & 0 & 0 \\ 0 & 0.000768905501178 & 0 \\ 0 & 0 & 6.88531064581e-05 \end{bmatrix} \quad (3)$$

Body 4

$$I_2 = \begin{bmatrix} 0.00037242266488 & 0 & 0 \\ 0 & 0.000356178538461 & 0 \\ 0 & 0 & 4.96474819141e-05 \end{bmatrix} \quad (4)$$

Τέλος, για τα σχετικά *screw axes* θα είναι ως γνωστόν:

$$S_i = [r_i, -r_i \times q_i]^T \quad (5)$$

Όπου στην (5) το διάνυσμα r_i το βρίσκουμε πηγαίνοντας στο πεδίο <axis xyz> = " .." και q_i μπορούμε να θεωρήσουμε το i -οστό διάνυσμα της 2^{ης} στήλης του πίνακα 2. Προκύπτουν τα εξής:

Joint	S_i
1	[0 0 1 0 0 0]
2	[0 1 0 -0.1775 0 0]
3	[0 1 0 -0.4945 0 0]
4	[0 1 0 -0.6965 0 0]
5	—

Ερώτημα 2: Δυναμικός προσομοιωτής

Στόχος του δυναμικού προσομοιωτή είναι η δυναμική μοντελοποίηση του ρομποτικού συστήματος. Παρακάτω παραθέτουμε/εξηγούμε τις σχετικές εξισώσεις.

Spatial Inertia Matrix

$G_b = \begin{bmatrix} I_b & 0 \\ 0 & M * I \end{bmatrix} \in R^{6 \times 6}(6)$, όπου I_b είναι το *rotational inertia matrix* για κάθε (το διορθώνω αυτό σε μελλοντική έκδοση), I ο 3×3 μοναδιαίος πίνακας (στην περίπτωση που έχουμε μόνο 1 σώμα και το μητρώο G_b εκφράζεται στο *Center of Mass* $\equiv$ *Body frame* και M είναι η μάζα του σώματος.

Adjoint τελεστής για τα twists

$[ad_v] = \begin{bmatrix} \hat{\omega} & 0 \\ \hat{v} & \hat{\omega} \end{bmatrix} (7)$, με το καπελάκι να αντιστοιχεί σε *skew-symmetric* πίνακα διάστασης 3×3

Εξίσωση *Inverse Dynamics* (για 1 *rigid body*)

$$F_b = G_b * V_b' - [ad_v]^T * G_b * V_b = [\tau_b \ f_b]^T, \text{ για κάθε link } (8)$$

Manipulator equation

$M(q) * \ddot{q} + C(q, \dot{q}) = \tau + F_{ext} \quad (9)$, στην οποία:

$M(q)$: Mass Matrix

$C(q, \ddot{q})$: Coriolis/Gravity Forces

τ : ροπές στα joints

Εμείς όμως έχουμε έναν βραχίονα με 4 *links* (5, αν θεωρήσει κάποιος και τη βάση), οπότε χρησιμοποιούμε την παρακάτω εξίσωση για τα *Inverse Dynamics*):

$$\tau = M(q) * \ddot{q}_d + C(q, \dot{q}) - F_{ext} \quad (10)$$

Στην προκειμένη περίπτωση θεωρούμε ότι δεν υφίστανται εξωτερικές δυνάμεις, συνεπώς:

$$F_{ext} = 0 \quad (11)$$

Συνάρτηση *Lagrangian*

$L = \sum_{i=1}^{\#of\ links} (K_i - P_i)$ (12), K_i και P_i είναι η κινητική και η δυναμική ενέργεια αντιστοίχως του i -οστού *link*

Αλγόριθμος *Inverse Dynamics Newton-Euler*

$M_{i,j}$: ο 4×4 πίνακας μετασχηματισμού για το *home configuration* (το οποίο ποιο είναι στην προκειμένη περίπτωση) από το *frame* $\{i\}$ στο *frame* $\{j\}$. Οπότε θα είναι:

$$M_i = M_{i,0} \text{ και } M_{i-1,i}^{-1} = M_{i-1}^{-1} * M_i \text{ και } M_{i,i-1} = M_i^{-1} * M_{i-1} \quad (13)$$

$T_{i,j}(q)$: ο 4×4 πίνακας μετασχηματισμού για οποιοδήποτε *configuration* από το *frame* $\{i\}$ στο *frame* $\{j\}$. Θα είναι:

$$T_{i-1,i}(q_i) = M_{i-1,i} * \exp(A_i * q_i) \quad (14)$$

και

$$T_{i,i-1}(q_i) = \exp(-A_i * q_i) * M_{i,i-1} \quad (15)$$

Twist για το *link* $\{i\}$ στο *frame* $\{i\}$:

$$V_i = [\omega_i \ v_i]^T \quad (16)$$

Wrench το οποίο μεταφέρεται από το *joint* i στο *link frame* $\{i\}$, εκπεφρασμένο στο *frame* $\{i\}$:

$$F_i = [\tau_i \ f_i]^T \quad (17)$$

Το δε *spatial matrix* για το *link* $\{i\}$ εκπεφρασμένο στο *frame* $\{i\}$

$G_i = \begin{bmatrix} I_i & 0 \\ 0 & m_i * I \end{bmatrix} \in R^{6 \times 6}$ (18), m_i είναι η μάζα του κάθε *link* (βλ. τον σχετικό πίνακα στο 1^ο ερώτημα)

Ο αλγόριθμος αυτός καθ'αυτός έχει ως εξής:

1) Υπολογισμός «ταχυτήτων» και «επιταχύνσεων»:

$$V_i = A_i * \dot{q}_i + [Ad_{T_{i,i-1}}] * V_{i-1} \quad (19)$$

$$Ad_T = \begin{bmatrix} R & 0 \\ \hat{p} & R \end{bmatrix} \quad (20) \text{ για } T = (R, p)$$

$$\dot{V}_i = A_i * \ddot{q}_i + [Ad_{T_{i,i-1}}] * \dot{V}_{i-1} + [ad_{V_i}] * A_i * \dot{q}_i \quad (21).$$

Οι εξισώσεις (15), (20), (21) αποτελούν το *forward pass*

Πριν παραθέσουμε τις σχετικές εξισώσεις για το *backward pass*, αφού στο *URDF* αρχείο, το *CoM* δεν ταυτίζεται με το εκάστοτε *body frame*, θα πρέπει να προβούμε στον παρακάτω μετασχηματισμό για το *spatial inertia matrix* για κάθε *link*:

$G'_i = [Ad_{T_{ba}}]^T * G_i * [Ad_{T_{ba}}]$ (22), T_{ba} είναι το *transformation matrix* από το *body frame* *a* στο *CoM frame* *b*. Πλέον μπορούμε να υπολογίσουμε τις «δυνάμεις» οι οποίες επιδρούν σε κάθε *link* μέσω των παρακάτω εξισώσεων για το *backward pass*:

$$F_i = [Ad_{T_{i+1,i}}]^T F_{i+1} + G'_i * \dot{V}_i - [ad_{V_i}]^T * G_i * V_i \quad (23)$$

(F_{n+1} είναι η δύναμη που ασκεί ο *end effector* στο περιβάλλον του)

$$\tau_i = F_i^T * A_i \quad (24)$$

Ο παραπάνω αλγόριθμος είναι πρόδηλο ότι πρόκειται για έναν αναδρομικό αλγόριθμο. Οι αναδρομικοί αλγόριθμοι ναι μεν είναι ως επί το πλείστον σχετικά απλή στο *implementation*, εντούτοις σε αρκετές περιπτώσεις συγκλίνουν με αργό ρυθμό. Γι'αυτό το λόγο, καταφεύγουμε στην αξιοποίηση ενός άλλου αλγόριθμου-«λιγότερο αναδρομικού».

Ορίζουμε λοιπόν τους παρακάτω πίνακες:

$$V_{all} = [V_1 \dots V_n]^T \in R^{6n} \quad (25) (n = 5 \text{ στο εν λόγω πρόβλημα})$$

$$F_{all} = [F_1 \dots F_n]^T \in R^{6n} \quad (26)$$

$$A_{all} = diag(A_1, A_2, \dots, A_n) \in R^{6n \times 6n} \quad (27)$$

$$G_{all} = diag(G_1, G_2, \dots, G_n) \in R^{6n \times 6n} \quad (28)$$

$$[ad_{V_{all}}] = diag([ad_{V_1}], [ad_{V_2}], \dots, [ad_{V_n}]) \quad (29)$$

$$[ad_{A_{all}*\dot{q}}] = diag([ad_{A_1*q_1}], [ad_{A_2*q_2}], \dots, [ad_{A_n*q_n}]) \quad (30) \in R^{6n \times 6n}$$

$$W(q) = \begin{bmatrix} 0 & 0 & \dots & 0 & 0 \\ [Ad_{T_{2,1}}] & 0 & \dots & 0 & 0 \\ 0 & [Ad_{T_{3,2}}] & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & \dots & [Ad_{T_{n,n-1}}] & 0 \end{bmatrix} \in R^{6n \times 6n} \quad (31)$$

$$L(q) = \begin{bmatrix} I & 0 & 0 & \dots & 0 \\ [Ad_{T_{2,1}}] & I & 0 & \dots & 0 \\ [Ad_{T_{2,1}}] & [Ad_{T_{3,2}}] & I & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ [Ad_{T_{n,1}}] & [Ad_{T_{n,2}}] & [Ad_{T_{n,3}}] & \dots & I \end{bmatrix} \in R^{6n \times 6n} \quad (32)$$

$$V_{base} = \begin{bmatrix} [Ad_{T_{1,0}}] * V_0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \in R^{6n} \quad (33)$$

$$\dot{V}_{base} = \begin{bmatrix} [Ad_{T_{1,0}}] * \dot{V}_0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \in R^{6n} \quad (34)$$

$$F_{tip} = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ [Ad_{T_{n+1,n}}]^T * F_{n+1} \end{bmatrix} \in R^{6n} \quad (35)$$

Επομένως, θα είναι:

$$V_{all} = L(q) * (A_{all} * \dot{q} + V_{base}) \quad (36)$$

$$\dot{V}_{all} = L(q) * (A_{all} * \ddot{q} - [ad_{A_{all}*\dot{q}}] * W(q) * V - [ad_{A_{all}*\dot{q}}] * V_{base} + \dot{V}_{base}) \quad (37)$$

$$F_{all} = L^T(q) * (G_{all} * \dot{V}_{all} - [ad_{V_{all}}]^T * G_{all} * V_{all} + F_{tip}) \quad (38)$$

$$\tau = A_{all}^T * F_{all} \quad (39)$$

Τέλος:

$$M(q) = A_{all}^T * L^T(q) * G_{all} * L(q) * A_{all} \quad (40)$$

$$c(q, \dot{q}) = -A_{all}^T * L^T(\dot{q}) * (G_{all} * L(q) * [ad_{A_{all} * \dot{q}}] * W(q) + [ad_{V_{all}}]^T * G_{all}) * L(q) * A_{all} * \dot{q} \quad (41)$$

$$g(q) = A_{all}^T * L^T(q) * G_{all} * L(q) * \dot{V}_{base} \quad (42)$$

Σημείωση: Στον σχετικό κώδικα κάποιοι όροι στις παραπάνω εξισώσεις για λόγους απλότητας/λειτουργικότητας του κώδικα παραλείπονται.

Ερώτημα 3 – Σχεδιασμός ελεγκτή

Η λογική του ελεγκτή ο οποίος χρησιμοποιείται έχει ως εξής

- 1) Υπολογισμός σφάλματος
- 2) Υπολογισμός δυνάμεων στον end-effector
- 3) Υπολογισμός joint accelerations
- 4) Ενημέρωση τιμών joint ταχυτήτων και γωνιών
- 5) Υπολογισμός joint torques

(Άρα δε χρησιμοποιούμε Inverse Kinematics)

Παρακάτω παρατίθενται οι σχετικές εξισώσεις:

$$\begin{aligned} e(t) \\ = q_d(t) \\ - q(t) \end{aligned} \quad (43), \text{ όπου } q_d \text{ είναι το επιθυμητό pose και } q(t) \text{ το πραγματικό}$$

$$f = K_p * e(t) + K_d * \dot{e}(t) + K_i * \int e(t) * dt \quad (44)$$

$$q_{des} = [q_x \ q_y \ q_z]^T \quad (45) \text{ το επιθυμητό pose του end - effector}$$

$$q_x = q_y = q_z = 0.2 * \sin(2\pi * 0.25 * t) \quad (46)$$

$$\begin{aligned} e(t) &= [e_x \ e_y \ e_z]^T \\ &= q_{des} - q_{actual} \quad (q_{actual} \text{ είναι το πραγματικό pose του end} \\ &\quad - \text{effector}) \end{aligned} \quad (47)$$

$$\ddot{q} = M^{-1} * (\tau - C - g) \quad (48)$$

$$\begin{aligned} \tau \\ = J_{half}(q)^T \\ * F \end{aligned} \quad (49), J_{half} \text{ είναι οι 3 πρώτες γραμμές του Ιακωβιανού πίνακα για τον end} \\ - \text{effector (γραμμικό κομμάτι)}.$$

$$\dot{q}(t + \Delta t) = \dot{q}(t) + a * \ddot{q}(t) * \Delta t \quad (50)$$

$$q(t + \Delta t) = q(t) + a * \dot{q}(t + \Delta t) * \Delta t \quad (51)$$

Ερώτημα 4 – Κώδικας (παρατηρήσεις)

- 1) Απαιτείται να γίνουν install οι παρακάτω βιβλιοθήκες
 - a) *numpy*
 - b) *matplotlib*
 - c) *scipy*
- 2) Ο κώδικας δημιουργήθηκε εξ'ολοκλήρου από το *Chat-GPT* αλλά με συνεχές (και πολύωρο) *prompting* του γράφοντος, εντούτοις τα αποτελέσματα δεν είναι και ιδιαίτερα ποιοτικά, κάτι το οποίο μπορεί να οφείλεται:
 - a) Σε λανθασμένη κατανόηση του προβλήματος
 - b) Σε κάποιο λάθος/παράλειψη στις εμπλεκόμενες εξισώσεις (σε κάποιες επείτηδες έγινε το παραπάνω μιας και σε διαφορετική περίπτωση ο κώδικας δεν έτρεχε)
 - c) Στις παραμέτρους/στον τύπο του ελεγκτή ο οποίος χρησιμοποιείται
 - d) Στις ελλειπείς γνώσεις προγραμματισμού του συντάκτη της παρούσης εργασίας
 - e) Στη μέθοδο ολοκλήρωσης (*Newton-Euler Inverse Dynamics Algorithm*)
 - f) Στο *time step* χρησιμοποιείται για τον αλγόριθμο ολοκλήρωσης
 - g) Σε λανθασμένη ανάγνωση/μεταφορά στον κώδικα των περιεχομένων του σχετικού *URDF* αρχείου
 - h) Στο γεγονός ότι έγινε προσπάθεια να γίνουν πολλά πράγματα *from scratch*, χωρίς να γίνει χρήση βιβλιοθηκών που θα διευκόλυναν σε σημαντικό βαθμό (πχ *Pinocchio*).
 - i) Στις παραμέτρους της επιθυμητής ημιτονοειδούς τροχιάς
 - j) Λανθασμένα *screw axes*
 - k) Σύγχυση ανάμεσα στο Mass Matrix *M* και στον πίνακα *M* ο οποίος χρησιμοποιείται στον αλγόριθμο Newton-Euler

κ.α

3) Ο κώδικας εκτελείται μέσω του αρχείου *main.py*